1.4 Polynomial regression

- what if our data is more complex than a straight line?
- surprisingly, we can use a linear model to fit *nonlinear data*
- a simple way to do this is to add powers of each feature as new features, then train a linear model on this extended set of features – this technique is called *polynomial regression*
- so far, we considered the linear regression setting described in (2), which allowed us to fit straight lines to data using maximum likelihood estimation
- however, straight lines are not sufficiently expressive when it comes to fitting more interesting data
- fortunately, linear regression offers us a way to fit nonlinear functions within the linear regression framework: since "linear regression" only refers to "linear in the parameters", we can perform an arbitrary nonlinear transformation $\phi(\boldsymbol{x})$ of the inputs $\boldsymbol{x}$ and then linearly combine the components of this transformation

1.4 Polynomial regression

- the corresponding linear regression model is:

$$\hat{y} = \boldsymbol{w}^\top \phi(\boldsymbol{x}), \tag{9}$$

  where $\phi : \mathbb{R}^d \to \mathbb{R}^p$ is a (nonlinear) transformation of the inputs $\boldsymbol{x}$ and $\phi_k : \mathbb{R}^d \to \mathbb{R}$ is the kth component of the feature vector ϕ
- note that the feature vector model parameters still appear only linearly
- note that when there are multiple features, polynomial regression is capable of finding relationships between features (which is something a plain linear regression model cannot do)
- this is made possible by the fact that we can also add all combinations of features up to the given degree
- for example, if there were two features a and b, we would not only add the features a^2, a^3, b^2, and b^3, but also the combinations ab, a^2b, and ab^2

1.4 Polynomial regression

- consider the dataset in Figure 13; the dataset consists of $n = 10$ pairs $(x^{(i)}, y^{(i)})$, where $x^{(i)} \sim \mathcal{U}[-5, 5]$ and

$$y^{(i)} = \sin\left(\frac{x^{(i)}}{5}\right) + \cos(x^{(i)}) + \epsilon,$$

where $\epsilon \sim \mathcal{N}(0, 0.2^2)$

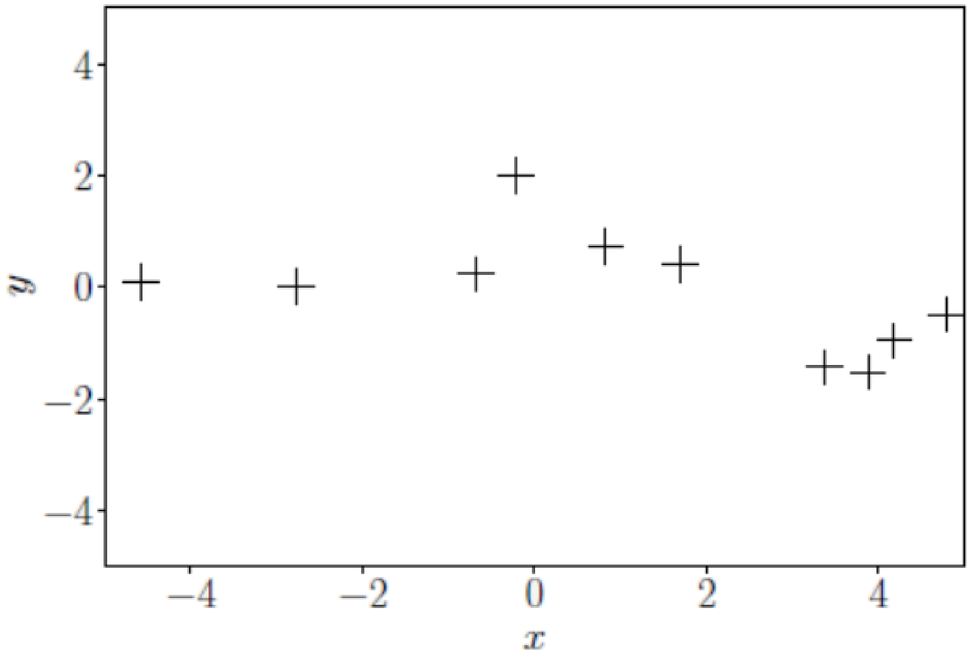


Figure 13: Polynomial regression: dataset consisting of $(x^{(i)}, y^{(i)})$ pairs, $i = 1, \ldots, 10$.

1.4 Polynomial regression

- assume that we use a polynomial model

$$\hat{y} = \boldsymbol{w}^\top \phi(x),$$

to fit it, where $\boldsymbol{w} \in \mathbb{R}^M$,

$$\phi(x) = \begin{bmatrix} 1 \\ x \\ x^2 \\ \vdots \\ x^{M-1} \end{bmatrix} \in \mathbb{R}^M.$$

- we can evaluate the quality of a model by computing the loss $\mathcal{L}$
- instead of using this squared loss, we often use the *root mean square error (RMSE)*:

$$\sqrt{\frac{1}{n}\sum_{i=1}^{n}(\hat{y}^{(i)} - y^{(i)})^2} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(\boldsymbol{w}^\top \phi(\boldsymbol{x}^{(i)}) - y^{(i)})^2},$$

which

- allows us to compare errors of datasets with different sizes
- has the same scale and the same units as the observed function values $y^{(i)}$

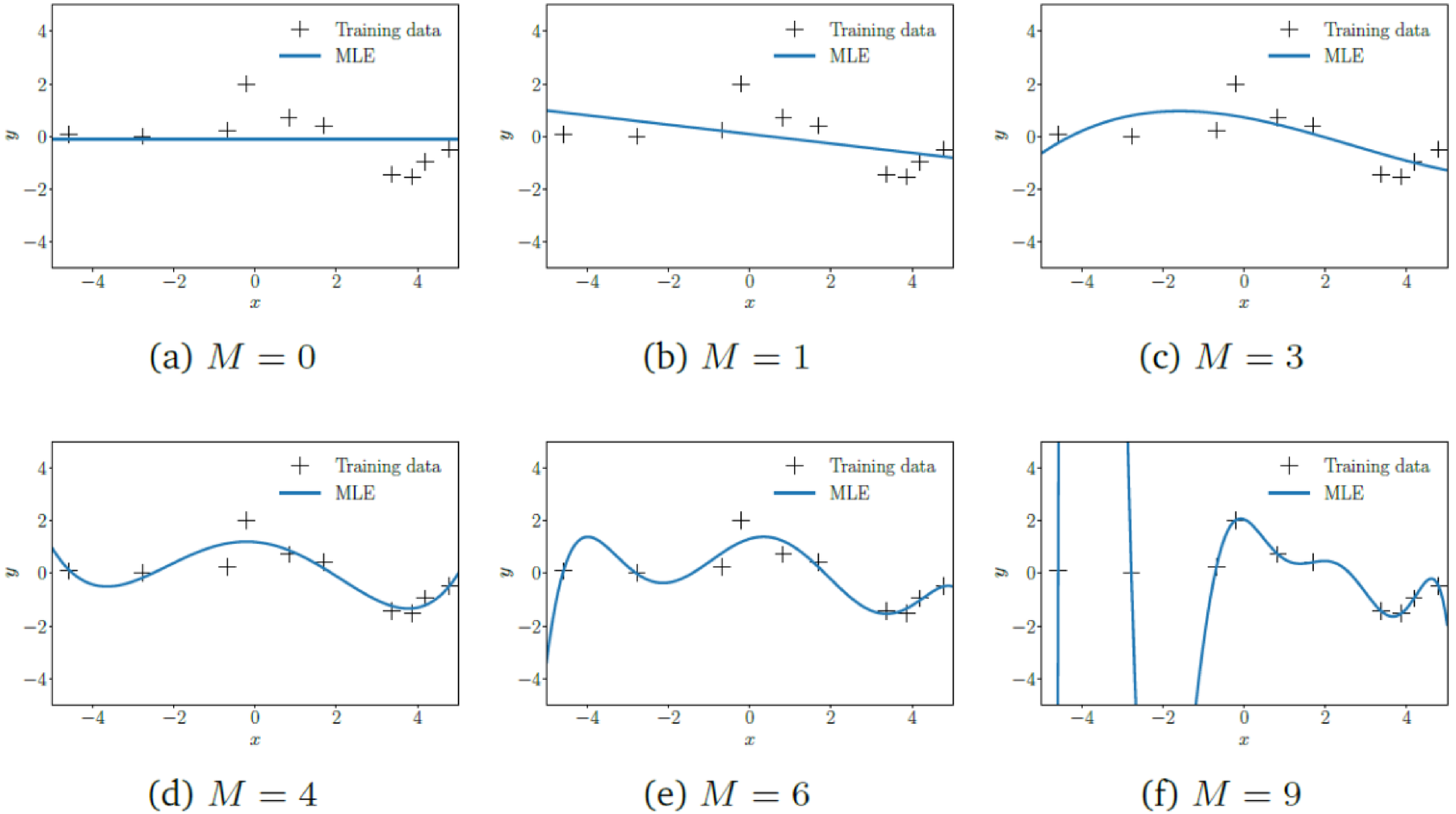
1.4 Polynomial regression

- in order to select the best model to fit our dataset, we can use the RMSE to determine the best degree of the polynomial by finding the polynomial degree M that minimizes the RMSE
- given that the polynomial degree is a natural number, we can perform a brute-force search and enumerate all (reasonable) values of M
- for a training set of size n it is sufficient to test $0 \leq M \leq n - 1$
- for $M < n$, the maximum likelihood estimator is unique
- for $M \geq n$, we have more parameters than data points, and would need to solve an underdetermined system of linear equations, so that there are infinitely many possible maximum likelihood estimators

1.4 Polynomial regression

- Figure 14 shows a number of polynomial fits determined by polynomial regression for the dataset from Figure 13 with $n = 10$ observations
- we notice that polynomials of low degree (e.g., constants ($M = 0$) or linear ($M = 1$)) fit the data poorly and, hence, are poor representations of the true underlying function
- for degrees $M = 3, \ldots, 6$, the fits look plausible and smoothly interpolate the data
- when we go to higher-degree polynomials, we notice that they fit the data better and better
- in the extreme case of $M = n - 1 = 9$, the function will pass through every single data point
- however, these high-degree polynomials oscillate wildly and are a poor representation of the underlying function that generated the data, such that we suffer from *overfitting*

1.4 Polynomial regression



(a) $M = 0$ (b) $M = 1$ (c) $M = 3$

(d) $M = 4$ (e) $M = 6$ (f) $M = 9$

Figure 14: Maximum likelihood fits for different polynomial degrees M.

1.4 Polynomial regression

- however, the goal is to achieve good *generalization* by making accurate predictions for new (unseen) data
- we obtain some quantitative insight into the dependence of the generalization performance on the polynomial of degree M by considering a separate test set comprising 200 data points generated using exactly the same procedure used to generate the training set
- as test inputs, we choose a linear grid of 200 points in the interval of $[-5, 5]$
- for each choice of M, we evaluate the RMSE for both the training data and the test data

1.4 Polynomial regression

- looking now at the *test error*, which is a qualitative measure of the generalization properties of the corresponding polynomial, we notice that initially the test error decreases; see Figure 15 (orange)
- for fourth-order polynomials, the test error is relatively low and stays relatively constant up to degree 5
- however, from degree 6 onward, the test error increases significantly, and high-order polynomials have very bad generalization properties
- in this particular example, this also is evident from the corresponding maximum likelihood fits in Figure 14

1.4 Polynomial regression

- note that the *training error* (blue curve in Figure 15) never increases when the degree of the polynomial increases
- in our example, the best generalization (the point of the smallest *test error*) is obtained for a polynomial of degree $M = 4$

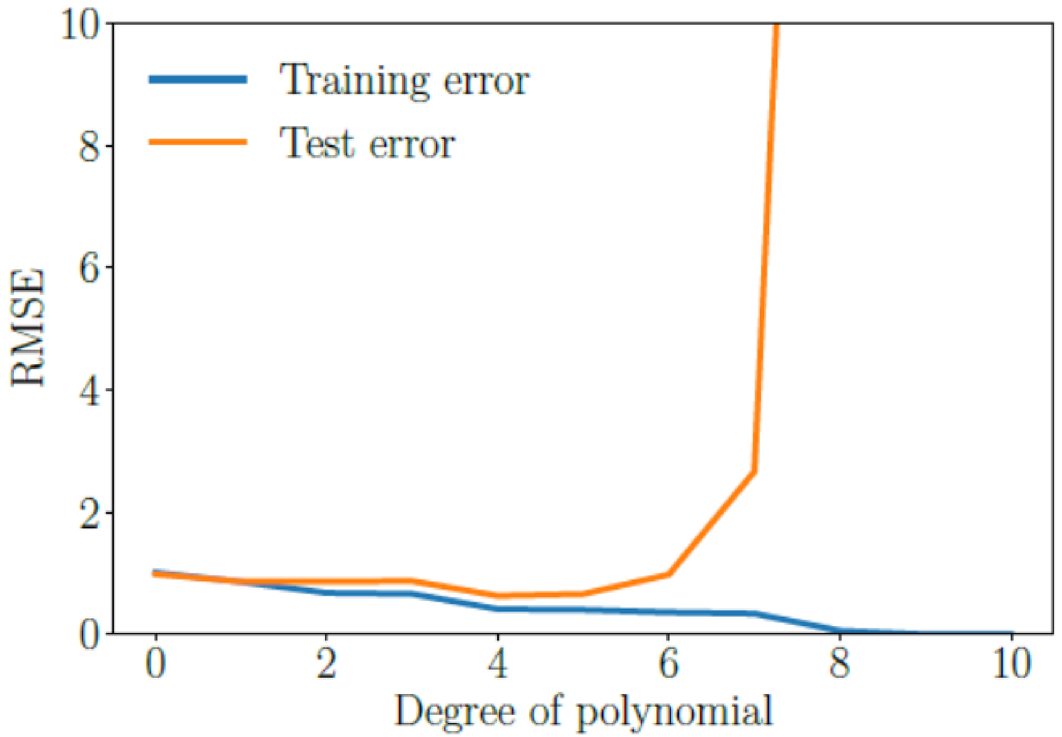


Figure 15: Training and test error.

1.4 Polynomial regression

- an important theoretical result of statistics and machine learning is the fact that a model's generalization error can be expressed as the sum of three very different errors:
  1. *Bias*. This part of the generalization error is due to wrong assumptions, such as assuming that the data is linear when it is actually quadratic. A high-bias model is most likely to underfit the training data.
  2. *Variance*. This part is due to the model's excessive sensitivity to small variations in the training data. A model with many degrees of freedom (such as a high-degree polynomial model) is likely to have high variance and thus overfit the training data.
  3. *Irreducible error*. This part is due to the noisiness of the data itself. The only way to reduce this part of the error is to clean up the data (e.g., fix the data sources, such as broken sensors, or detect and remove outliers).
- increasing a model's complexity will typically increase its variance and reduce its bias
- conversely, reducing a model's complexity increases its bias and reduces its variance
- this is why it is called the *bias/variance trade-off*